Day 05: Movie Catalog Service (MongoDB, APIs & Data Handling)

By the end of this session, learners will be able to:

- Integrate NoSQL database using MongoDB
- Design scalable document-based schemas
- Implement CRUD and search APIs
- Apply pagination and sorting mechanisms
- Develop reviews and ratings system
- Understand file upload handling
- Implement exception handling and response standardization

Service Overview: Movie Catalog Service

The Movie Catalog Service is responsible for:

- Managing movie data
- Providing search and filtering capabilities
- Handling user reviews and ratings
- Supporting media (poster/image) uploads

Academic Insight:

Unlike relational systems, NoSQL services focus on flexible schema design and high read performance, making them ideal for content-heavy applications like movie platforms.

MongoDB Integration

Technology: MongoDB

Why MongoDB?

- Schema flexibility
- JSON-like document storage
- High scalability
- Optimized for read-heavy workloads

Spring Boot Integration Components

- Spring Data MongoDB
- MongoTemplate / Repository abstraction

Configuration Concept

```
spring:
  data:
    mongodb:
      uri: mongodb://localhost:27017/cineverse
```

Movie Schema Design

Document-Based Modeling: Unlike tables, MongoDB uses documents (JSON-like structures).

Movie Schema (Conceptual)

```
{
  "title": "Inception",
  "genre": ["Sci-Fi", "Thriller"],
  "rating": 8.8,
  "language": "English",
  "duration": 148,
```

```
  "releaseDate": "2010-07-16",
  "posterUrl": "image.jpg"
}
```

Design Considerations

- Use arrays for genres
- Embed frequently accessed data
- Avoid excessive normalization

Insight:

MongoDB schema design follows denormalization principles to optimize read performance.

CRUD APIs

CRUD operations form the foundation of any service.

Core APIs

Operation	**Endpoint**	**Method**
Create Movie	/movies	POST
Get Movies	/movies	GET
Get by ID	/movies/{id}	GET
Update movie	/movies/{id}	PUT
Delete Movie	/movies/{id}	DELETE
Create other endpoints as needed. E.g., categories, etc.		

Example Controller

```
@PostMapping("/movies")
public Movie createMovie(@RequestBody Movie movie) {
   return movieService.save(movie);
}
```

Academic Insight:

CRUD operations should follow RESTful principles, ensuring predictable API behavior.

Search APIs

Search functionality is critical in content-based platforms.

Search Types

- By title
- By genre
- By rating

Example

```
@GetMapping("/movies/search")
public List<Movie> search(@RequestParam String title) {
   return movieService.searchByTitle(title);
}
```

Advanced Concept

- Regex-based search
- Text indexing

Insight:

Efficient search requires indexing strategies to reduce query time complexity.

Pagination & Sorting

Why Needed?

- Prevent large data loads
- Improve performance
- Enhance user experience

Concept

Pagination divides results into pages:

- Page number
- Page size

Example

```
PageRequest.of(0, 10, Sort.by("rating").descending());
```

Benefits

- Reduced response size
- Faster API performance

Academic Insight:

Pagination is essential for scalable API design, especially in high-data systems.

Reviews & Ratings APIs

Purpose

- Capture user feedback
- Improve recommendation systems

Data Model

```
{
 "movieId": "123",
 "userId": "456",
 "rating": 4,
 "review": "Great movie!"
}
```

APIs

Operation	Endpoint	Method
Add Review	/reviews	POST
Fetch reviews	/reviews/{movieId}	GET

Design Insight

Ratings can be:

- Aggregated (average rating)
- Stored per user

Academic Insight:

Review systems introduce user-generated data, requiring validation and moderation strategies.

File Upload Basics

Purpose: Upload movie posters/images

Concept

- Multipart file upload
- Store file locally or cloud

Example

```
@PostMapping("/upload")
public String upload(@RequestParam MultipartFile file) {
   return file.getOriginalFilename();
}
```

Storage Options

- Local storage
- Cloud storage (AWS S3, etc.)

Insight:

File handling introduces challenges like storage management, security, and scalability.

Exception Handling

Common types of Exceptions

- Resource not found
- Invalid input
- Server errors

Example

```
@ExceptionHandler(RuntimeException.class)
public ResponseEntity<?> handleError() {
   return ResponseEntity.badRequest().body("Error occurred");
}
```

Academic Insight:

Exception handling ensures system robustness and fault tolerance.

Response Standardization

Standard responses ensure consistency across APIs.

Standard Format

```
{
 "status": "success",
 "message": "Movie fetched successfully",
 "data": {}
}
```

Benefits

- Easier frontend integration
- Better debugging
- Consistent API contracts

Insight:

Standardization aligns with API design best practices and improves maintainability.

Integration with Other Services

- Communicates via API Gateway (Day 04)
- Secured via JWT
- Used by frontend (Day 02)

Expected Learning Outcomes

Students will be able to:

- Design NoSQL schemas for real-world systems
- Implement scalable CRUD and search APIs
- Apply pagination and sorting
- Develop review and rating systems
- Handle file uploads
- Implement robust error handling

Deliverables

- Movie Catalog Service fully implemented
- MongoDB integrated
- CRUD APIs functional
- Search APIs implemented
- Pagination and sorting enabled
- Reviews and ratings module completed
- File upload functionality added
- Standardized API responses

Viva Questions (Without Answers)

1. What is MongoDB and how is it different from relational databases?
2. Explain document-based schema design.
3. What are the advantages of NoSQL databases?
4. What are CRUD operations in REST APIs?
5. How does search functionality work in MongoDB?
6. What is pagination and why is it important?
7. Explain sorting in API responses.
8. How are reviews and ratings handled in a system?
9. What are the challenges in designing a review system?
10. What is file upload and how is it implemented?
11. What is exception handling in Spring Boot?
12. Why is response standardization important?
13. How does indexing improve search performance?

14. What are best practices for API design?
15. How does MongoDB improve scalability?
16. What is denormalization and why is it used?
17. What are the limitations of NoSQL databases?
18. How do microservices interact with the Movie Service?